

# 04 · Synchronisation und Ressourcenverwaltung

Race Conditions, Mutexe, Deadlocks und Priority Inversion · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

**Folie**

1 / 53

## Willkommen auf der dunklen Seite der Echtzeit!

- Herzlich willkommen zum 4. Termin unserer Vorlesung
- Letzte Woche: FreeRTOS installiert und Präemption gefeiert
- Heute: Warum genau diese Präemption das System zerstören kann
- Leitfrage: Was passiert, wenn zwei Tasks gleichzeitig auf dieselbe Hardware zugreifen?
- Werkzeuge: Semaphoren und Mutexe, um das Chaos zu verhindern

## Notizen

Willkommen zur vierten Vorlesung! Letzte Woche: FreeRTOS und Präemption als Feature für harte Deadlines. Heute die gefährliche Seite: Asynchronität korrumpiert Daten und kann das System lahmlegen. Fokus: Synchronisation in Echtzeitsystemen.

**Folie**

2 / 53

## Agenda für heute

1. Recap: Die Macht der Präemption
2. Race Conditions und geteilte Ressourcen
3. Assembler-Ebene: Warum `counter++` crasht
4. Lösungsansätze: Spinlocks vs. Interrupt-Sperren vs. OS-Blocking
5. Semaphoren und Mutexe
6. Deadlocks, Priority Inversion und Mars Pathfinder (PIP)

## Notizen

Problem-Analyse zuerst: Race Conditions und Schuld der Präemption. Dann C auf Assembler-Ebene. Alte Konzepte (Spinlock, IRQ-Sperre) und warum sie im RTOS tödlich sind. Saubere Lösung: Semaphoren und Mutexe. Nach der Pause: Deadlocks und Prioritätsinversion.

## Recap: Die Macht der Präemption

- Präemptives Scheduling: Das OS (SysTick) kann einer laufenden Task jederzeit die CPU entreißen
- Die unterbrochene Task merkt davon nichts
- Pause exakt am aktuellen Program Counter (PC)
- CPU-Register werden auf den Stack gerettet
- Diese Asynchronität ist gewollt — und extrem gefährlich

### Notizen

Präemption: SysTick, PendSV rettet Register, andere Task läuft. Die unterbrochene Task kann nicht sagen „Warte, Satz zu Ende schreiben“. Gefahr: beide Tasks beschreiben dasselbe „Blatt Papier“ — Shared Resource.

## Motivation: Der Kampf um den UART

- Szenario auf dem STM32:
  - Task\_Temp (Prio Normal) sendet: "Temperatur: 22 Grad\n"
  - Task\_Hum (Prio High) sendet: "Feuchtigkeit: 40 %\n"
- Beide rufen `printf()` auf — byteweise Schreiben auf dasselbe UART-Register

### Notizen

Alltäglich: Temperatur- und Feuchtigkeits-Task teilen sich den UART und `printf`. Feuchtigkeit hat höhere Prio (Timer-getriggert). Was erscheint auf der Konsole?

## Das visuelle Chaos

- Task\_Temp beginnt: `> Temp`
- SysTick feuert — Task\_Hum (High) präemptiert
- Task\_Hum schreibt fertig: `> TempFeuchtigkeit: 40 %`
- Task\_Hum schläft; Task\_Temp schreibt den Rest
- Ergebnis: `> TempFeuchtigkeit: 40 %eratur: 22 Grad` — Buchstabensalat

## Notizen

Temperatur-Task schreibt T-e-m-p, dann präemptiert die Feuchtigkeits-Task und schreibt in dasselbe Register. Danach Rest „eratur“ — Datenstrom korrumpiert.

## Folie

6 / 53

### Das Problem geteilter Ressourcen

- UART ist eine Shared Resource (geteilte Ressource)
- Unabgesprochener Zugriff mehrerer Tasks — Datenkorruption
- Typische Shared Resources in Embedded Systems:
  - Peripherie-Register (UART, SPI, I2C)
  - Hardware (Displays, Motoren)
  - Globale Variablen (Arrays, Structs, Fehlerzähler)
  - Dynamischer Speicher (`malloc/free`)

## Notizen

Buchstabensalat ist nur die sichtbare Form. Sobald zwei Threads asynchron dieselbe Ressource nutzen: Absturz oder Datenverlust. Auch globale Arrays: halb neu, halb alt — logischer Fehler.

## Folie

7 / 53

### Lernziele des heutigen Tages

1. Race Condition auf Assembler-Ebene erklären
2. Unterschied Spinlocks vs. OS-Blocking verstehen
3. CMSIS-API für Semaphoren und Mutexe kennen
4. Wissen, warum Mutexe Ownership besitzen und Semaphoren nicht
5. Priority Inversion und Priority Inheritance kennen

## Notizen

Race Condition auf CPU-Ebene, nicht nur als Buzzword. Semaphor vs. Mutex gezielt einsetzen. Prioritätsinversion: unwichtige Task reißt System mit; FreeRTOS löst mit Priority Inheritance.

## Was ist eine Race Condition?

- Race Condition (Wettlaufsituation):
  - Verhalten hängt unvorhersehbar von der zeitlichen Reihenfolge ab
  - Code „funktioniert meistens“, crasht aber selten (Heisenbug)
- Auslöser: asynchrone Präemption während einer Datenmanipulation

### Notizen

Ergebnis hängt davon ab, wer den Wettlauf gewinnt. In 99 % der Fälle zufällig ok. Mit Breakpoints verschwindet der Fehler — klassischer Heisenbug.

## Das simpelste Code-Beispiel

- Globale Variable: `int fehler = 0;`
- Task A (Low) und Task B (High) melden beide einen Fehler
- Beide rufen `fehler++;` auf
- War der Wert 5, erwarten wir danach 7

```
// Globale Variable (Shared Resource)
volatile uint32_t error_counter = 5;

void Task_A() {
    // ...
    error_counter++;
    // ...
}

void Task_B() {
    // ...
    error_counter++;
    // ...
}
```

### Notizen

Klassischer Heisenbug: zwei Tasks inkrementieren `error_counter` von 5. Logik sagt 7 — unter ungünstigem Timing steht am Ende 6. Ein Fehler wurde verschluckt.

## Der Blick unter die Haube (Assembler)

- In C wirkt `counter++` wie ein Befehl
- ARM ist Load/Store: kein direktes „Erhöhe-RAM“
- Compiler übersetzt in Read-Modify-Write (3 Befehle):
  1. LDR: Wert aus RAM in Register ( $R0 = 5$ )
  2. ADD: Register in der ALU erhöhen ( $R0 = 6$ )
  3. STR: Register zurück in den RAM schreiben
- Was, wenn Präemption genau zwischen LDR und STR zuschlägt?

### Notizen

C ist eine Illusion: Load/Store-Architektur. Interrupt nach jedem Assembler-Befehl möglich. Kritisch: PendSV zwischen ADD und STR.

## Folie

## Visualisierung der Datenkorruption



- Task B schreibt 6; Task A überschreibt danach mit veralteter 6 — die 7 ist verloren

### Notizen

Task A: LDR 5, ADD  $R0 = 6$ , dann SysTick. Register auf Stack. Task B liest RAM noch 5, speichert 6. Task A setzt mit STR seine 6 — Erhöhung von B vernichtet.

## Atomarität (Die Lösung des Problems)

- Ein Code-Block muss atomar ausgeführt werden
- Atomar (griech. *atomos* = unteilbar): keine Unterbrechung, kein sichtbarer Zwischenzustand
- Sobald Task A ausliest, darf keine andere Task lesen/schreiben, bis Task A gespeichert hat
- Frage: Wie erzwingen wir Atomarität in C?

### Notizen

Read-Modify-Write wurde zerrissen. Atomarität: LDR/ADD/STR unteilbar. Mehrere Wege in C — nächste Folien.

## Definition der Begriffe

- Kritische Variable (Shared Resource): Daten, die geschützt werden müssen
- Kritischer Abschnitt (Critical Section): Code, der atomar auf die Ressource zugreift
- Ziel: nur die Critical Section absichern — nicht die ganze Task

```
void Task_A() {  
    // ... Unkritischer Code ...  
  
    // --- START CRITICAL SECTION ---  
    error_counter++;  
    // --- END CRITICAL SECTION ---  
  
    // ... Unkritischer Code ...  
}
```

### Notizen

Daten vs. Code trennen: Shared Resource = Variable/Register; Critical Section = zugreifender Code. Nicht die ganze Task atomar machen — sonst kein Multitasking.

## Schutzidee 1: Der Spinlock (Polling)

- Idee aus der Bare-Metal-Welt: Flag `lock`
- Vor Zugriff: prüfen, ob `lock == 0`, dann auf 1 setzen
- Frage: Warum ist das im RTOS eine Katastrophe?

```
volatile int lock = 0;

void Task_A() {
    while(lock == 1); // Spinning
    lock = 1;

    error_counter++;

    lock = 0;
}
```

### Notizen

Spinlock: while-Schleife bis frei, dann sperren. Wartende Task „dreht sich im Kreis“. Warum in Single-Core-RTOS Garantie für Totalausfall?

## Warum Spinlocks im RTOS crashen

- Spinlocks verstoßen gegen RTOS-Regeln
- CPU verbrennen: wartende Task bleibt Running (nicht Blocked), 100% Last
- Deadlock-Szenario:
  1. Task L (Low) setzt `lock = 1`
  2. Task H (High) präemptiert und spinnt auf `lock`
  3. H gibt CPU nie ab — L kommt nie zu `lock = 0`
- System friert ein; Watchdog schlägt zu

### Notizen

OS sieht Spinning als Arbeit. High-Prio spinnt, Low-Prio kommt nie zum Entsperren — klassischer Deadlock auf Single-Core.

## Schutzidee 2: Interrupts sperren

- Präemption kommt von Hardware-IRQs — also IRQs abschalten?
- Cortex-M: `__disable_irq()` / `__enable_irq()`
- In FreeRTOS: `taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()`
- Ist das eine gute Idee?

```
void Task_A() {  
    __disable_irq();  
  
    error_counter++; // atomar  
  
    __enable_irq();  
}
```

### Notizen

Ohne Interrupts keine Präemption — `counter++` sicher. FreeRTOS bietet fertige Critical-Makros. Klingt perfekt — ist es nicht.

## Die Grenzen der Interrupt-Sperre

- Vorteil: extrem schnell (oft 1 Taktzyklus)
- Nachteil: zerstört harte Echtzeit
  - 5 ms IRQ-Sperre: kein UART-RX, kein Motor-Timer, kein Not-Halt
- Best Practice:
  - Nur für extrem kurze Zugriffe (eine Variable)
  - Niemals für lange `printf()` oder Delays

### Notizen

Nuklearoption: Welt steht still, Latenzgarantien weg. Erlaubt: drei Werte kopieren oder Zähler++. `printf` mit gesperrten IRQs = No-Go.

### Die „richtige“ Lösung: OS-Blocking

- Nicht `while()` und nicht Hardware-Sperren
- Lösung über den RTOS-Scheduler
- An das OS: „Schloss belegt? Dann blockiere mich, bis es frei ist.“
- Wartende Task → Blocked (0% CPU)
- Freigabe weckt die wartende Task
- Werkzeuge: Semaphoren und Mutexe

#### Notizen

Schloss vom Scheduler verwalten lassen. Blocked statt Spinning: keine CPU-Verschwendung, kein Spinlock-Deadlock, Interrupts laufen weiter.

### Zusammenfassung Block 1

- Asynchrone Präemption + Shared Variables → Race Conditions
- C-Befehl wird zu `LDR/ADD/STR` — Datenverlust möglich
- Critical Sections müssen atomar abgesichert sein
- Spinlocks: CPU-Brand und Deadlocks
- Interrupt-Sperren: Latenzgarantien zerstört
- Saubere Lösung: OS-Blocking (nächster Block: Semaphoren)

#### Notizen

Heisenbugs auf Assembler-Ebene verstanden. Spinlocks und lange IRQ-Sperren sind Gift. Fragen zur Hardware-Ebene, bevor Block 2 (Semaphoren) beginnt?

## Was ist ein Semaphore?

- Griechisch: „Zeichenträger“ (*sema* = Zeichen, *pherein* = tragen)
- Historisch: Eisenbahn-Signalfügel (frei / Halt)
- IT (Edsger W. Dijkstra, 1965):
  - Vom OS verwaltete Integer-Variable (Zähler)
  - Zwei atomare Operationen: Acquire (Nehmen) und Release (Freigeben)

### Notizen

Semaphor = Zeichenträger, Eisenbahn-Analogie. Dijkstra 1965: Zähler mit Acquire/Release, exklusiv vom Scheduler verwaltet.

## Typ 1: Counting Semaphore (Zähl-Semaphor)

- Zähler von 0 bis  $N$
- Analogie: Parkhaus mit  $N = 50$  Plätzen
- Acquire: Auto rein  $\rightarrow$  Zähler sinkt
- Zähler = 0: Schranke zu, weitere Tasks Blocked
- Release: Auto raus  $\rightarrow$  Zähler steigt, Wartende werden geweckt
- Einsatz: begrenzte Ressourcen (z. B. Pool aus 3 Verbindungen)

### Notizen

Parkhaus-Analogie: Acquire = einfahren, Release = ausfahren. Bei 0 blockiert das OS die nächste Task, bis wieder Platz ist.

## Typ 2: Binary Semaphore (Binärer Semaphor)

- Spezialfall mit Max-Wert  $N = 1$
- Zähler nur 0 (belegt) oder 1 (frei)
- Analogie: Toilettenschlüssel an der Tankstelle
- In FreeRTOS/CMSIS meist binär genutzt

## Notizen

Parkhaus mit einem Platz / ein Schlüssel. In der Mikrocontroller-Praxis ~95 % binäre Semaphoren.

## Folie

23/53

### CMSIS-API für Semaphoren

```
// 1. Definition & Initialisierung
osSemaphoreId_t mySemHandle;
mySemHandle = osSemaphoreNew(1, 1, NULL); // (Max-Count, Initial

// 2. Warten (Acquire)
osSemaphoreAcquire(mySemHandle, osWaitForever);

// 3. Freigeben (Release)
osSemaphoreRelease(mySemHandle);
```

## Notizen

`osSemaphoreNew(1, 1, NULL)`: binär, Schlüssel frei. Timeout: `osWaitForever` oder z. B. 100 Ticks. Danach `osSemaphoreRelease`.

## Folie

24/53

### Was exakt passiert bei Acquire / Release?

Acquire (Schlüssel holen)

- OS prüft Zähler
- $\geq 1$ : Zähler  $-1$ , Task läuft weiter
- $== 0$ : Task Running  $\rightarrow$  Blocked

Release (Schlüssel zurück)

- OS erhöht Zähler um 1
- Wartende Task in Blocked-Liste?
- Ja: höchstprioritätig  $\rightarrow$  Ready (ggf. Präemption)

## Notizen

Acquire: Zähler  $> 0 \rightarrow$  abziehen und weiter; sonst Blocked. Release: erhöhen, ggf. Ready und sofortiger Kontextwechsel.

## Der Hauptanwendungsfall: Signalisierung

- Binärer Semaphor in FreeRTOS selten zur Absicherung
- Primär: Signalisierung (Einweg-Kommunikation)
- Deferred Interrupt Processing:
  - ISR muss kurz bleiben
  - ISR ruft nur `Release()`
  - Wartende Task (`Acquire`) übernimmt die Arbeit

### Notizen

Zur Variablenabsicherung kommt der Mutex. Semaphoren: Signal aus ISR an Task. Deferred Interrupt Processing = zeitversetzte Verarbeitung in Task-Kontext.

## Sequenzdiagramm: Signalisierung aus der ISR



### Notizen

Task wartet, CPU idle. ISR nur Release, dann Task Ready/Running, Acquire setzt Zähler auf 0. Elegante Anbindung asynchroner Hardware ohne lange ISR.

### Semaphoren haben kein „Ownership“

- Kein Besitzrecht: OS merkt sich nicht, wer Acquire gemacht hat
- Task A: Acquire; Task B: Release — erlaubt!
- Für Signalisierung (ISR → Task) genial
- Für Variablen-/UART-Schutz: Sicherheitsrisiko
- Fremdtask kann Schloss versehentlich „aufbrechen“

#### Notizen

Kein Ownership: ISR darf Release ohne Acquire. Für UART-Schutz gefährlich — wir brauchen ein Schloss mit Besitzer.

### Die Rettung: Der Mutex (Mutual Exclusion)

- Mutex = gegenseitiger Ausschluss
- Konzeptionell binärer Semaphor, aber mit Ownership
- Eiserne Regel: nur die sperrende Task darf entsperren
- Fremde Task/ISR wird abgewiesen
- Perfekt für Critical Sections

#### Notizen

Mutual Exclusion + Ownership im TCB. Falsches Release → Fehlercode, Schloss bleibt zu. Sicher gegen Fremdeinwirkung.

### CMSIS-API für Mutexe

```
// 1. Definition
osMutexId_t uartMutex;
uartMutex = osMutexNew(NULL);

void PrintTask() {
    for(;;) {
        osMutexAcquire(uartMutex, osWaitForever);

        // --- CRITICAL SECTION ---
        printf("Temperatur: 22 Grad\n");
        // --- END CRITICAL SECTION ---

        osMutexRelease(uartMutex);
        osDelay(500); // Ticks; bei 1000 Hz = 500 ms
    }
}
```

### Notizen

API ähnlich Semaphor. Vor `printf`: Acquire; danach Release. Buchstabensalat vom UART-Beispiel ist damit behoben.

### Folie

30/53

### Sonderfälle in der C-Programmierung

- Was ist an diesem Code tödlich?
- Antwort: Mutex bleibt für immer verriegelt
- Früher oder später: Deadlock im System
- Achtung bei frühem `return` / `break` in Critical Sections

```
osMutexAcquire(myMutex, osWaitForever);

if (sensor_error == 1) {
    return; // Release fehlt!
}

error_counter++;
osMutexRelease(myMutex);
```

## Notizen

Klassiker: Acquire, dann früher Return ohne Release — Schlüssel verloren, alle anderen stapeln sich vor der Tür.

## Folie

31 / 53

### Best Practices für Mutexe

- So kurz wie möglich in der Critical Section bleiben
- Niemals in einer Critical Section `osDelay()` aufrufen
- Niemals Mutex-Acquire in einer Hardware-ISR
  - ISR kann nicht Blocked werden → Crash
- In ISRs: Semaphoren oder Queues, keine Mutexe

## Notizen

Kurz halten: sperren, kopieren, entsperren. `osDelay` im Mutex = Todsünde. ISR darf nicht schlafen — nur Semaphor/Queue. Als Nächstes: Deadlocks, Priority Inversion und Mars Pathfinder.

## Folie

32 / 53

### Rekursive Mutexe

- Was passiert, wenn eine Task eine Funktion aufruft, die einen Mutex sperrt, und diese wiederum denselben Mutex anfordert?
- Normaler Mutex: Self-Deadlock — die Task blockiert sich selbst
- Rekursiver Mutex:
  - OS merkt sich den Owner
  - Erneutes Acquire durch denselben Owner: `count++` statt Blockade
  - Freigabe erst, wenn Release genauso oft wie Acquire (`count == 0`)
- Nutzen: verschachtelte Bibliotheken (z. B. Dateisysteme)

## Notizen

Spezialfall rekursiver Mutex: Bibliotheksfunktion A sperrt, ruft Hilfsfunktion B auf, B sperrt denselben Mutex. Normaler Mutex → Self-Deadlock. Rekursiv: Owner erkannt, Counter hoch; Release muss ebenso oft folgen.

## Zusammenfassung: Mutex vs. Binärer Semaphore

| Eigenschaft           | Binärer Semaphore                      | Mutex (Mutual Exclusion)            |
|-----------------------|----------------------------------------|-------------------------------------|
| Hauptzweck            | Signalisierung (ISR → Task)            | Absicherung (Ressourcenschutz)      |
| Ownership             | Nein                                   | Ja (nur Besitzer entsperrt)         |
| Verwendung in ISR?    | Ja (Release weckt Task)                | Nein (Systemabsturz)                |
| Priority Inheritance? | Nein                                   | Ja (dazu gleich mehr)               |
| Gefahr                | Fremdtask öffnet Schloss versehentlich | Priority Inversion (ohne OS-Schutz) |

### Notizen

Semaphore: Event aus ISR an Task, kein Ownership. Mutex: persönliches Schloss für Shared Resources. Klausurregel: Array/UART absichern → immer Mutex.

## Die Katastrophe: Wenn Locks böse werden

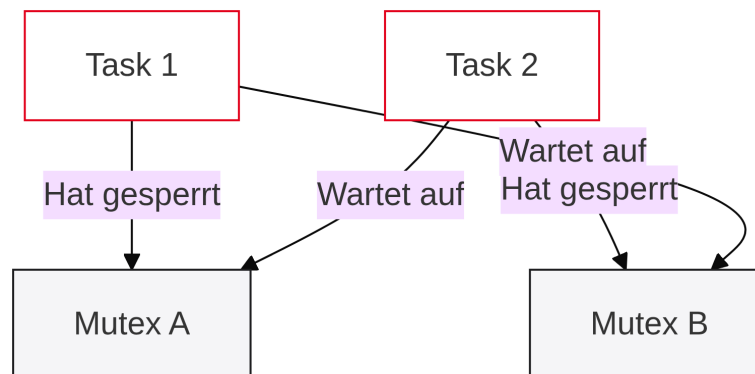
- Mutexe schützen Daten — erzeugen aber zwei gefürchtete Fehlerbilder:
  1. Deadlocks (Verklemmung)
  2. Priority Inversion (Prioritätsinversion)
- Oft erst nach Wochen im Feldtest sichtbar
- Zerstören das Vertrauen in Echtzeitgarantien

### Notizen

Falscher Umgang mit Schlössern: Deadlocks und Prioritätsinversionen. Am Schreibtisch selten, unter Last und asynchronen Interrupts kritisch.

## Der Deadlock (Verklemmung)

- Deadlock: Tasks warten zirkulär aufeinander — niemand kommt weiter
- Beispiel:
  - Task 1 sperrt Mutex A
  - Präemption → Task 2 sperrt Mutex B
  - Task 2 will Mutex A → Blocked
  - Task 1 will Mutex B → Blocked
- Ergebnis: System Freeze



### Notizen

Zwei Ressourcen (SPI = Mutex A, Display = Mutex B). Task 1 hält A, Task 2 hält B; beide wollen den anderen Mutex → Zirkelschluss, kein Aufwachen mehr.

## Vermeidung von Deadlocks

- Regel 1: Keine geschachtelten Mutexe
  - Niemals einen zweiten Mutex anfordern, solange einer gehalten wird
- Regel 2: Strikte Anforderungsreihenfolge
  - Wenn Schachtelung unvermeidbar: alle Tasks fordern Mutexe in derselben globalen Reihenfolge an (erst A, dann B)
- So kann Task 2 niemals B sperren, solange A schon belegt ist

### Notizen

Beste Praxis: ein Schloss, Critical Section, freigeben, dann nächstes. Bei zwei Schlössern: globale Hierarchie. Task 2 scheitert an A und geht Blocked, bevor sie B berührt — Task 1 hat freie Bahn.

## Der subtile Feind: Priority Inversion

- Deadlock: System steht — offensichtlich
- Priority Inversion: System läuft weiter, aber Echtzeitgarantien der wichtigsten Task sind zerstört
- Setup:
  - Task L (Low), Task M (Medium), Task H (High)
  - Ein Mutex, den nur L und H teilen
  - M braucht den Mutex nicht

### Notizen

Prioritätsinversion ist subtiler als Deadlock: keine Freeze-Anzeige, aber High wird ausgebremst. Low und High teilen den Bus-Mutex; Medium fasst ihn nie an. Nicht verwechseln mit Starvation: Inversion entsteht durch Mutex-Halten + präemptierende Mittelprio.

## Die Anatomie der Inversion (Schritt 1 und 2)

- Task L ist Running und sperrt den Mutex (Critical Section)
- Task H wacht auf und präemptiert L
- Task H: Acquire scheitert → Blocked (L hält das Schloss)
- Scheduler gibt CPU an L zurück, damit L freigeben kann
- Bis hierhin: normale, gewollte Blockade

### Notizen

High will den Bus, Low hält ihn → High geht Blocked, Low rechnet weiter. Physikalisch korrekt: Low muss fertig werden dürfen.

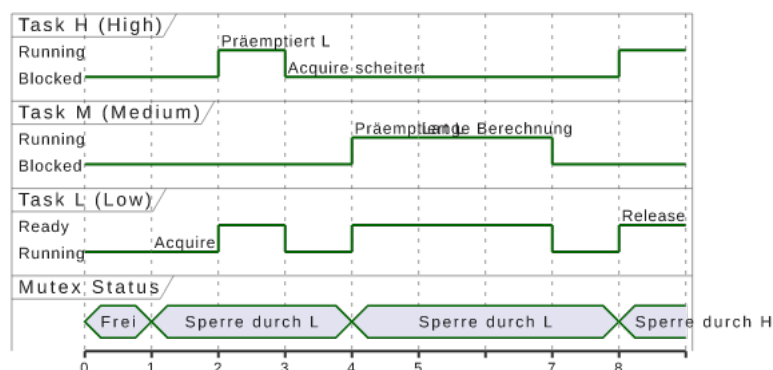
### Die Anatomie der Inversion (Schritt 3 und 4)

- Task L rechnet noch (hält den Mutex)
- Task M wacht auf und präemptiert L ( $Prio_M > Prio_L$ )
- Task M braucht den Mutex nicht — lange Berechnung
- Drama:
  - M blockiert L
  - L kann den Mutex nicht freigeben
  - H (wichtigste Task!) wartet unbegrenzt in der Queue

### Notizen

Medium verdrängt Low nach prioritätsbasiertem Scheduling. High hängt Blocked auf Lows Schloss — eine mittelpriore Task blockiert indirekt die höchstpriore. Das ist Priority Inversion, nicht bloßes Verhungern.

### Visualisierung der Priority Inversion



### Notizen

Von T=4 bis T=7 rechnet Medium; High bleibt Blocked. Prioritäten logisch umgekehrt: Medium agiert de facto als Höchstprio. Latenz von High hängt von unbeteiligter Task ab — Albtraum für harte Echtzeit.

### Warum ist das so katastrophal?

- Verzögerung von Task H ist nicht mehr durch eine WCET begrenzt
- Jede aufwachende Medium-Prio-Task verlängert die Wartezeit von H
- Harte Echtzeitgarantien brechen (Deadline Miss)
- Genau dieses Muster trat 1997 auf dem Mars auf

### Notizen

Normale Blockade: WCET der Critical Section planbar. Inversion: Rechenzeiten aller Medium-Tasks addieren sich zur Wartezeit von High — Latenz ungebunden. Kein akademisches Konstrukt: NASA Pathfinder 1997.

### Die reale Katastrophe: Mars Pathfinder (1997)

- 4. Juli 1997: Pathfinder landet mit Rover Sojourner auf dem Mars
- Wenige Tage später:
  - Bordcomputer friert ein
  - Hardware-Watchdogs lösen aus
  - Wiederholte Resets (Reset-Loop)
- Ein Totalverlust drohte

### Notizen

Euphorie nach erfolgreicher Landung, dann Panik im JPL: sporadische Freezes, Watchdog-Resets. Bug-Jagd im Simulator auf der Erde für ein System 300 Millionen km entfernt.

### Die Architektur des Pathfinder

- Betriebssystem: VxWorks (präemptives RTOS)
- Tasks (stark vereinfacht):
  - Bus-Manager (Task H): höchste Prio, zeitkritischer Informations-Bus
  - ASI/MET (Task L): niedrige Prio, Wetter-/Atmosphärendaten auf den Bus
  - Kommunikation (Task M): mittlere Prio, formatiert Funkpakete zur Erde
- Bus durch Mutex geschützt; M fasst den Mutex nicht an

## Notizen

Klassisches L/M/H-Setup: Wetter-Task schreibt über den Bus-Mutex, Bus-Manager braucht denselben Bus blitzschnell, Kommunikations-Task belastet die CPU ohne den Mutex.

## Folie

44 / 53

### Was auf dem Mars passierte

- ASI/MET (Low) sperrt den Bus-Mutex
- Bus-Manager (High) wacht auf, Acquire scheitert → Blocked
- Kommunikation (Medium) präemptiert Low und rechnet lange
- Watchdog: Sicherheits-Timer erwartet regelmäßiges Lebenszeichen des Bus-Managers
- Bus-Manager bleibt Blocked → Timer läuft ab → Hardware-Reset

## Notizen

Exakt unser Inversions-Szenario. Medium hält Low vom Freigeben ab; Watchdog glaubt das OS tot und resettet hart.

## Folie

45 / 53

### Die Lösung: Wie rettete die NASA die Sonde?

- VxWorks hatte bereits einen Schutz — standardmäßig deaktiviert (CPU-Overhead sparen)
- NASA lud über  $\approx 300$  Millionen km Funkstrecke einen C-Patch hoch
- Patch aktivierte ein Flag bei der Mutex-Erstellung
- Feature-Name: Priority Inheritance Protocol (PIP)

## Notizen

PIP war 1997 schon im OS, aber aus Performance-Gründen aus. Ein winziger Config-Patch per Funk — System stabil. Prioritätsvererbung war die Lösung.

### Das Priority Inheritance Protocol (PIP)

- Prioritätsvererbung: gezielter Eingriff des RTOS-Schedulers
- Regel: Wenn High einen von Low gehaltenen Mutex anfordert
  - dann erbt Low temporär die Priorität von High
- Low wird künstlich geboostet, bis Release

#### Notizen

OS erkennt: wichtige Task wartet vor dem Schloss. Es boostet den Owner (Low) auf High-Prio, damit die Critical Section so schnell wie möglich endet.

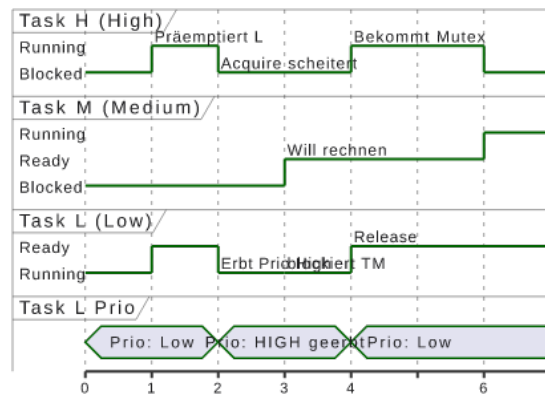
### Wie PIP das Mars-Problem löste

- Task L hält den Mutex; Task H blockiert bei Acquire
- PIP: L erhält temporär Prio High
- Task M wacht auf — kann L nicht mehr präemptieren ( $Prio_L$  jetzt High)
- L beendet Critical Section, ruft Release → Prio fällt zurück auf Low
- H übernimmt den Mutex und läuft

#### Notizen

Wetter-Task erbt Bus-Manager-Prio; Medium prallt ab. Nach Release fällt die geliehene Prio weg, High nimmt den Mutex — Watchdog gerettet.

## Visualisierung: Priority Inheritance („Die Prio-Beule“)



### Notizen

Vierte Zeile: Prio-Beule bei T=2. Medium bei T=3 Ready, prallt an geerbter High-Prio ab. Nach Release fällt Prio zurück — Inversion gebannt.

### Folie

49/53

## Achtung: Mutex vs. Semaphore in FreeRTOS

- Warum die strikte Trennung Binärer Semaphore vs. Mutex?
- Wichtigste FreeRTOS-/CMSIS-Regel:
  - `osMutexNew`: Priority Inheritance automatisch aktiv
  - `osSemaphoreNew`: niemals Priority Inheritance (kein Ownership)
- Niemals einen Semaphore zur Absicherung von UART oder Arrays nutzen

### Notizen

Beide wirken wie 0/1-Schalter — der Unterschied ist PIP. Semaphore kennt keinen Owner und kann keine Priorität vererben. UART mit Semaphore = Pathfinder-Risiko auf dem MCU.

## Laborvorbereitung: Chaos am UART

- Ziel: Priority Inversion auf dem STM32 provozieren und heilen
- Schritt 1 — Das Chaos:
  - Zwei Tasks (Task1\_Normal, Task2\_High)
  - Beide senden per `printf()` einen langen String
  - Keine Absicherung
- Beobachtung: Terminal zeigt zerrissenen String-Salat

### Notizen

Labor wie NASA: ungesicherte UART-Ausgabe, SysTick präemptiert — Buchstabensalat wie im UART-Motivationsbeispiel.

## Labor-Schritt 2: Die Absicherung

- CMSIS-Mutex anlegen (`osMutexNew`)
- `printf()` in Acquire/Release-Blöcke packen

```
void Task1_Normal() {  
    for(;;) {  
        osMutexAcquire(uartMutex, osWaitForever);  
        printf("Task 1 sendet in Ruhe Daten...\n");  
        osMutexRelease(uartMutex);  
  
        osDelay(100); // Ticks; bei 1000 Hz = 100 ms  
    }  
}
```

- Beobachtung: Strings im Terminal sind sauber

### Notizen

Critical Section um `printf`: Scheduler legt wartende Tasks Blocked. Console wieder lesbar. CMSIS: `osDelay` in Ticks — analog VL3, nicht `pdMS_TO_TICKS`.

### Labor-Schritt 3: Die Sabotage (Inversion)

- Mutex durch binären Semaphor ersetzen (kein PIP!)
- `Task3_Medium` hinzufügen (Prio zwischen Normal und High)
- Medium: `while (1)` ohne Delay (CPU-Fresser)
- Beobachtung:
  - Terminal verstummt / System hängt
  - Medium blockiert Low (hält Semaphor) → Priority Inversion: High bleibt ungebunden verzögert

#### Notizen

Semaphor statt Mutex = PIP aus. Heavy-Math-Task aus VL3 in der Mitte: klassische Priority Inversion (nicht Starvation im Scheduler-Sinn) — High wartet auf den Semaphor, Low hält ihn, Medium verdrängt Low.

### Labor-Schritt 4: Die Heilung

- Semaphor zurück zu `osMutexId_t` / `osMutexNew`
- Neu flashen
- Beobachtung:
  - Terminal sendet wieder
  - OS boostet Low temporär (Priority Inheritance)
  - Medium wird beiseitegeschoben, High gerettet
- Go für das Labor!

#### Notizen

Wie die NASA: eine API-Änderung heilt das System. FreeRTOS-Boost bringt Low durch die Critical Section. Wer das live sieht, hat Synchronisation in Echtzeit verstanden — viel Erfolg!